

Universidad Mariano Galvez de Guatemala

Facultad de Ingeniería en Sistemas de la información y ciencias de la computación

Carrera: Ingeniería en Sistemas

Carrera: Programación III



Tarea XIII
BFS y DFS

Nombre:

Carnet

Jorge Eduardo Espel Melgar

9941-24-10596

Erick Ruben Hernández Chávez

9941-24-24182

Fecha: 31/05/2026



Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Nombre:	Serge Eduardo Espartero Helyar	Carné:	9941-24-10596
Nombre:	CAROL RUBEN HERNANDEZ CANO	Carné:	9941-24-24182
Nombre:		Carné:	

Carrera	Ingeniería en Sistemas	Carrera	9941
Asignatura	Programación III	Curso	022
Ciclo	Quinto Ciclo	Jornada	Domingos
Catedrático	Ing. MBA David Alvarez	Fecha	24/05/2026
Semestre	Primer Semestre	Sección	B

Objetivo

Aplicar recorridos de grafos mediante BFS y DFS, representando gráficamente los grafos, resolviendo recorridos manualmente y comparando una implementación en C++ puro contra una implementación en Java con Collections Framework.

Parte 1 – Recorridos manuales de grafos

Instrucciones

Para cada ejercicio, el grupo debe:

1. Dibujar el grafo.
2. Escribir la lista de adyacencia.
3. Realizar el recorrido BFS desde el nodo indicado.
4. Realizar el recorrido DFS desde el nodo indicado.
5. Indicar el orden de visita.

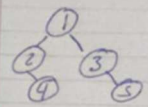
Cuando exista más de un vecino disponible, deben visitar los nodos en orden ascendente o alfabético.

Ejercicio 1 1-2 1-3 2-4 3-5 Inicio: 1	Ejercicio 2 1-2 1-4 2-3 4-5 5-6 Inicio: 1	Ejercicio 3 A-B A-C B-D B-E C-F Inicio: A
Ejercicio 4 1-2 2-3 3-4 4-5 Inicio: 1	Ejercicio 5 1-2 1-3 1-4 2-5 4-6 Inicio: 1	Ejercicio 6 A-B B-C C-D A-E E-F Inicio: A
Ejercicio 7 1-2 1-3 2-4 2-5 3-6 3-7 Inicio: 1	Ejercicio 8 A-B A-C B-D C-D D-E Inicio: A	Ejercicio 9 1-2 2-3 3-1 3-4 4-5 Inicio: 1
Ejercicio 10 A-B A-D B-C D-E C-F E-F Inicio: A	Ejercicio 11 1-2 1-5 2-3 2-4 5-6 6-7 Inicio: 1	Ejercicio 12 A-B A-C B-E C-D D-E E-F Inicio: A
Ejercicio 13 1-2 1-3 2-4 4-6 3-5 5-6 6-7 Inicio: 1	Ejercicio 14 A-B B-C C-D D-A A-E E-F Inicio: A	Ejercicio 15 1-2 1-3 2-5 3-4 4-6 5-6 6-7 7-8 Inicio: 1
Ejercicio 16 1-2 1-3 2-4 2-5 3-6 5-7 6-7 7-8 8-9 Inicio: 1	Dibuje el grafo. Escriba la lista de adyacencia. Realice el recorrido BFS desde el nodo 1. Realice el recorrido DFS desde el nodo 1. Responda: • ¿En qué momento (nivel) BFS llega al nodo 9? • ¿DFS visita el nodo 9 antes o después que BFS? • ¿Cuál recorrido considera más eficiente en este caso y por qué?	

Parte 1: Recorridos manuales de grafos

Sección I

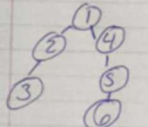
1



```
graph TD; 1((1)) --- 2((2)); 1 --- 3((3)); 2 --- 4((4)); 3 --- 5((5))
```

BFS = 1, 2, 3, 4, 5
DFS = 1, 2, 4, 3, 5

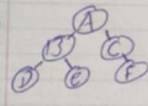
2



```
graph TD; 1((1)) --- 2((2)); 1 --- 4((4)); 2 --- 3((3)); 4 --- 5((5)); 4 --- 6((6))
```

BFS = 1, 2, 4, 3, 5, 6
DFS = 1, 2, 3, 4, 5, 6

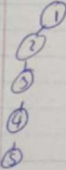
3



```
graph TD; A((A)) --- B((B)); A --- C((C)); B --- D((D)); B --- E((E)); C --- F((F))
```

BFS = A, B, C, D, E, F
DFS = A, B, D, E, C, F

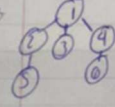
4



```
graph TD; 1((1)) --- 2((2)); 2 --- 3((3)); 3 --- 4((4)); 4 --- 5((5))
```

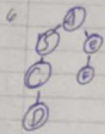
BFS = 1, 2, 3, 4, 5
DFS = 1, 2, 3, 4, 5

5

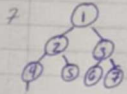


```
graph TD; 1((1)) --- 2((2)); 1 --- 4((4)); 2 --- 3((3)); 4 --- 5((5)); 4 --- 6((6))
```

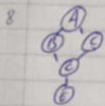
BFS = 1, 2, 3, 4, 5, 6
DFS = 1, 2, 5, 3, 4, 6



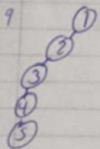
BFS = A, B, E, C, F, D
DFS = A, B, C, D, E, F



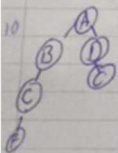
BFS = A, B, C, D, E
DFS = A, B, D, E, C



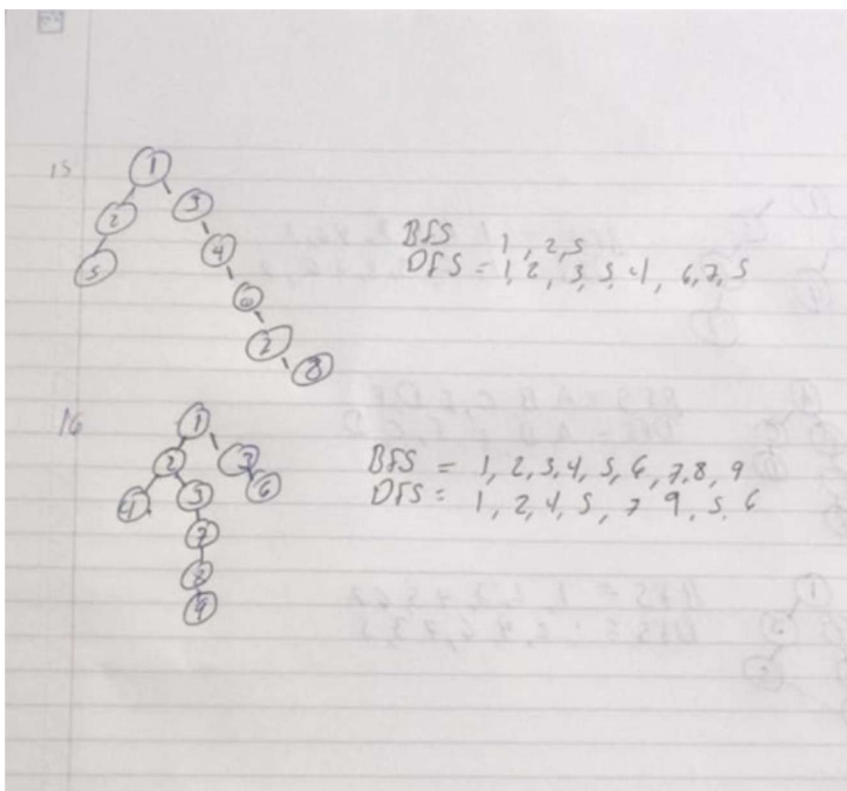
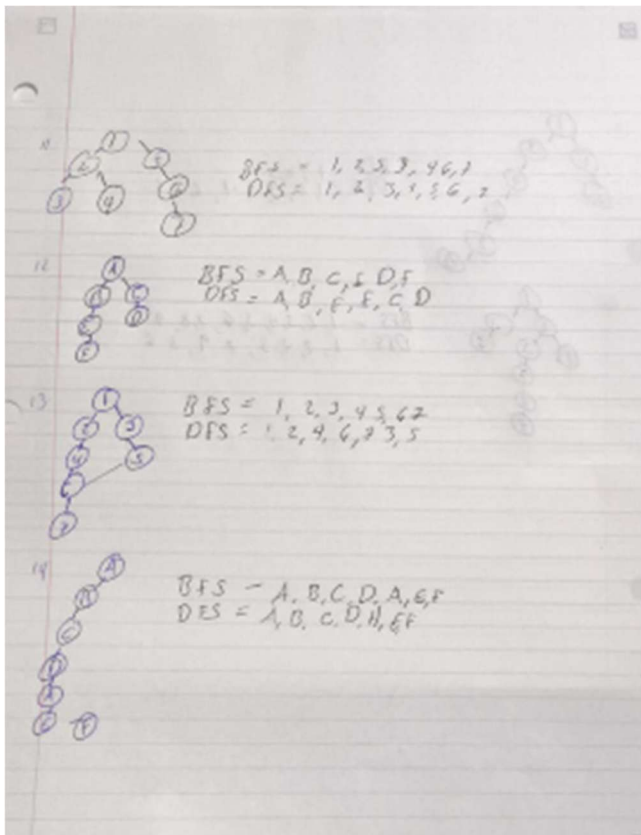
BFS = A, B, C, D, E
DFS = A, B, D, E, C



BFS = 1, 2, 3, 4, 5
DFS = 1, 2, 3, 4, 5

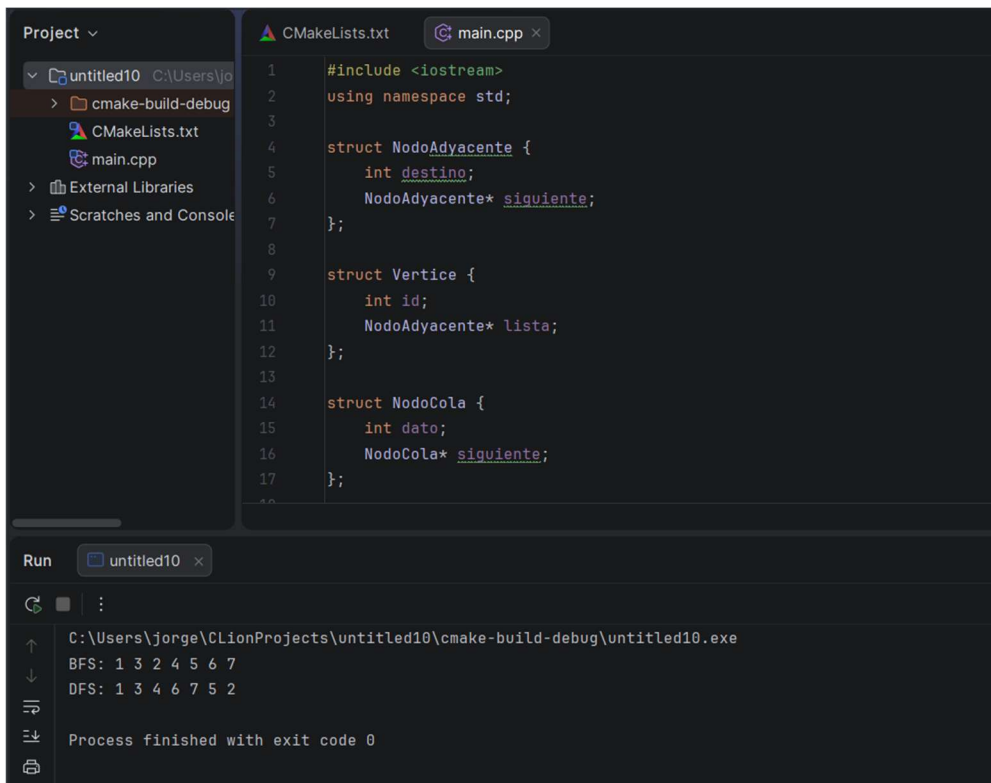


BFS = A, B, C, E, F
DFS = A, B, C, F, E



Parte 2: Comparación técnica C++ vs Java

Ejecución en C++:



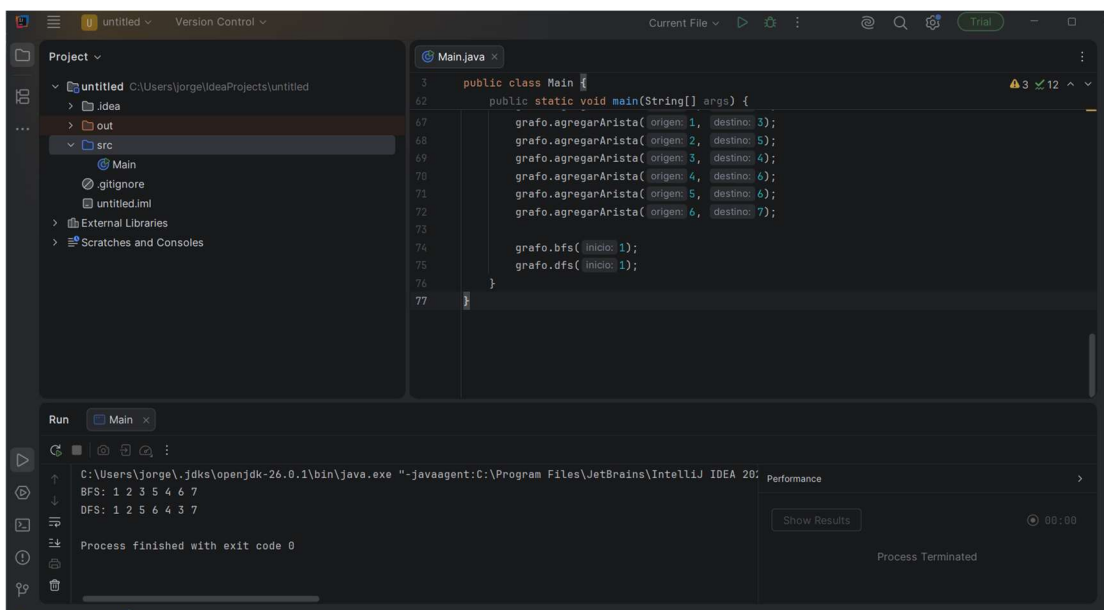
The screenshot shows the CLion IDE interface. The left sidebar displays the project structure for 'untitled10', including 'cmake-build-debug', 'CMakeLists.txt', and 'main.cpp'. The main editor window shows the 'main.cpp' file with the following C++ code:

```
1 #include <iostream>
2 using namespace std;
3
4 struct NodoAdyacente {
5     int destino;
6     NodoAdyacente* siguiente;
7 };
8
9 struct Vertice {
10     int id;
11     NodoAdyacente* lista;
12 };
13
14 struct NodoCola {
15     int dato;
16     NodoCola* siguiente;
17 };
18
```

Below the editor, the 'Run' tab is active, showing the execution of 'untitled10.exe'. The output console displays the following results:

```
C:\Users\jorge\CLionProjects\untitled10\cmake-build-debug\untitled10.exe
BFS: 1 3 2 4 5 6 7
DFS: 1 3 4 6 7 5 2
Process finished with exit code 0
```

Ejecución java:



The screenshot shows the IntelliJ IDEA IDE interface. The left sidebar displays the project structure for 'untitled', including 'idea', 'out', 'src', and 'Main'. The main editor window shows the 'Main.java' file with the following Java code:

```
3 public class Main {
62     public static void main(String[] args) {
67         grafo.agregarArista( origen: 1, destino: 3);
68         grafo.agregarArista( origen: 2, destino: 5);
69         grafo.agregarArista( origen: 3, destino: 4);
70         grafo.agregarArista( origen: 4, destino: 6);
71         grafo.agregarArista( origen: 5, destino: 6);
72         grafo.agregarArista( origen: 6, destino: 7);
73
74         grafo.bfs( inicio: 1);
75         grafo.dfs( inicio: 1);
76     }
77 }

```

Below the editor, the 'Run' tab is active, showing the execution of 'Main'. The output console displays the following results:

```
C:\Users\jorge\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 202...
BFS: 1 2 3 5 4 6 7
DFS: 1 2 5 6 4 3 7
Process finished with exit code 0
```

On the right side of the Run tab, there is a 'performance' section with a 'Show Results' button and a 'Process Terminated' status.

Análisis técnico requerido:

1. ¿Qué estructuras se implementaron manualmente en C++?

En C++ se implementaron manualmente:

- Lista de adyacencia mediante NodoAdyacente.
- Cola para BFS mediante NodoCola y la clase Cola.
- Vértices del grafo mediante Vertice.
- Manejo de memoria dinámica usando new y delete.

Esto fue necesario porque no se permitió utilizar estructuras predefinidas como vector, queue o stack.

2. ¿Qué estructuras ya existen en Java?

En Java se utilizaron estructuras ya incorporadas en la biblioteca estándar:

- HashMap
- ArrayList
- Queue
- LinkedList
- HashSet

Estas estructuras facilitan la implementación del grafo y reducen la cantidad de código necesario.

3. ¿Cuál código es más largo?

El código en C++ quedó más largo porque hubo que construir más cosas manualmente.

4. ¿Cuál código es más fácil de entender?

El código en Java es más fácil de entender porque:

- Tiene menos líneas.
- No utiliza punteros.
- Las estructuras de datos ya están definidas.
- La sintaxis es más simple y legible.

5. ¿Cuál lenguaje exige mayor control del programador?

C++ exige mayor control porque el programador debe:

- Administrar la memoria.
- Crear y destruir objetos dinámicamente.
- Evitar fugas de memoria.
- Implementar estructuras básicas desde cero.

Java automatiza gran parte de estas tareas.

6. ¿Qué ventajas tiene C++ en este caso?

- Mayor control sobre los recursos.
- Mejor rendimiento.
- Menor consumo de memoria si está bien implementado.
- Permite comprender mejor cómo funcionan internamente las estructuras de datos.

7. ¿Qué ventajas tiene Java en este caso?

- Desarrollo más rápido.
- Menos errores relacionados con memoria.
- Código más corto.
- Bibliotecas integradas muy completas.
- Mantenimiento más sencillo.

8. ¿Qué sucede si en C++ no se libera memoria correctamente?

Se producen:

- Fugas de memoria (Memory Leaks).
- Consumo innecesario de RAM.
- Disminución del rendimiento.
- Posibles fallos del programa.

En Java esto normalmente no ocurre porque existe el Garbage Collector.

9. ¿Por qué BFS necesita una cola?

Porque BFS trabaja por niveles y la cola mantiene el orden correcto de recorrido.

10. ¿Por qué DFS puede resolverse con recursión?

Porque DFS va profundizando nodo por nodo y la recursión ayuda a guardar automáticamente el camino recorrido.

Parte 3 Video:

https://drive.google.com/file/d/1wvoXRh3MXyiprq2COJkn_XPbU9DDXIJM/view?usp=sharing